

Custom Deep Learning Framework and Autoencoder Implementation

NAME	ID
Nour Emad Rabea	2101443
Abdulrahman Khaled	2101779
Mohamed Reda	2100459

Submitted To:

Dr. Hossam Hassan

Eng. Abdallah Mohammed

1. Introduction

While Part 1 focused on building the custom, NumPy-based neural network framework from scratch to master its core mechanics, this section presents Part 2 of the project, shifting the focus towards advanced applications and rigorous benchmarking.

Here, the framework's scalability and practical capability are demonstrated by building a Denoising Autoencoder for Fashion-MNIST image reconstruction. Furthermore, the quality of the learned representations is validated by utilizing the extracted latent space features to train a supervised Support Vector Machine (SVM) classifier. Finally, to definitively validate its mathematical integrity and efficiency, the custom framework is rigorously benchmarked against an industry-standard library (TensorFlow/Keras) under identical architectural constraints.

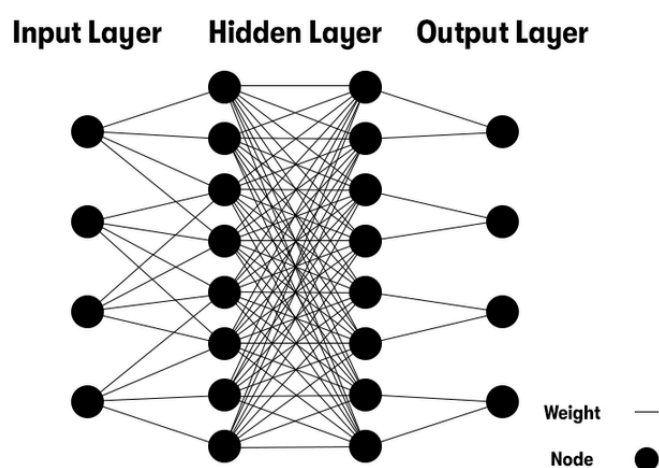


Figure 1: Neural Network Architecture — Input, Hidden, and Output Layers

2. Objectives

Evaluating Unsupervised Representation Learning

To design and implement a Denoising Autoencoder using the custom framework, evaluating its capacity to autonomously compress high-dimensional Fashion-MNIST data into a compact latent space and achieve high-fidelity image reconstruction against input corruption.

Validating Feature Transferability

To analyze the robustness and mathematical quality of the learned latent space representations by deploying the trained encoder as a feature extractor, feeding these compressed embeddings into a Support Vector Machine (SVM) to evaluate downstream supervised classification performance.

Computational Performance Benchmarking

To execute a rigorous comparative analysis between our custom handcrafted framework and an industry-standard library (TensorFlow/Keras), benchmarking critical metrics such as implementation complexity, training convergence time, and final reconstruction error under identical network architectures.

3. Denoising Autoencoder (Fashion-MNIST)

A Denoising Autoencoder was built using Fashion-MNIST to demonstrate the framework's scalability, forcing the network to learn compressed features by reconstructing clean images from corrupted inputs.

3.1 Data Preprocessing and Noise Injection

Images were flattened into 784-dimensional vectors and normalized to [0, 1]. Gaussian noise (factor: 0.2) was injected into the dataset, followed by clipping to maintain valid pixel ranges.

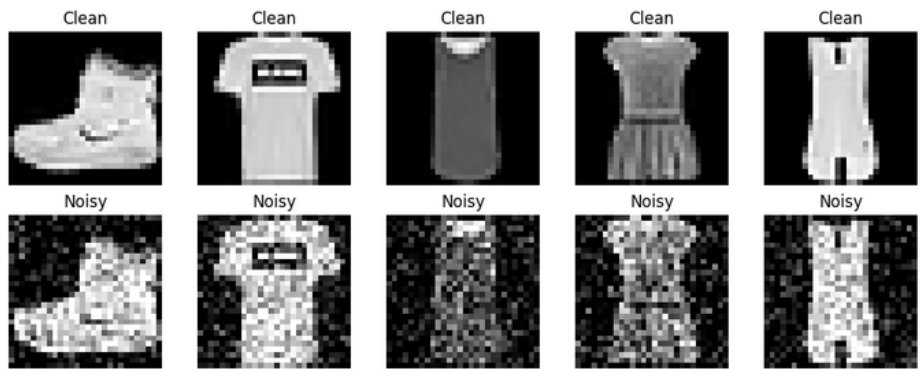


Figure 2: Clean vs. Noisy Fashion-MNIST samples — top row shows original clean images; bottom row shows the same images after Gaussian noise injection (factor 0.2).

3.2 Network Architecture

The model employs a symmetrical bottleneck design for dimensionality reduction. The Encoder progressively compresses the 784-pixel input into a highly compact 32-dimensional latent space, utilizing dropout regularization to prevent noise memorization. Conversely, the Decoder reconstructs the original image from this compressed representation, applying a Sigmoid activation at the final layer to ensure valid pixel intensities between 0 and 1.

Component	Layers	Activation
Encoder	784 → 128 → 64 → 32	ReLU + Dropout
Latent Space	32 dimensions	—
Decoder	32 → 64 → 128 → 784	ReLU + Sigmoid

3.3 Training and Results

The network was trained for 50 epochs (batch size: 128) using SGD (LR: 0.05, Momentum: 0.9). By mapping noisy inputs to clean targets, the model converged efficiently, achieving a final MSE loss of 0.0274.

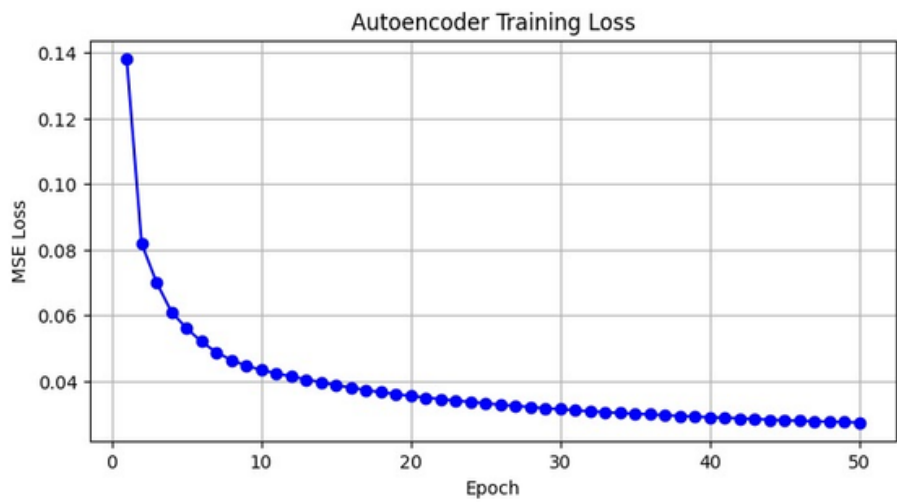


Figure 3: Autoencoder training loss curve over 50 epochs — loss drops steeply in the first 10 epochs then plateaus near 0.0274.

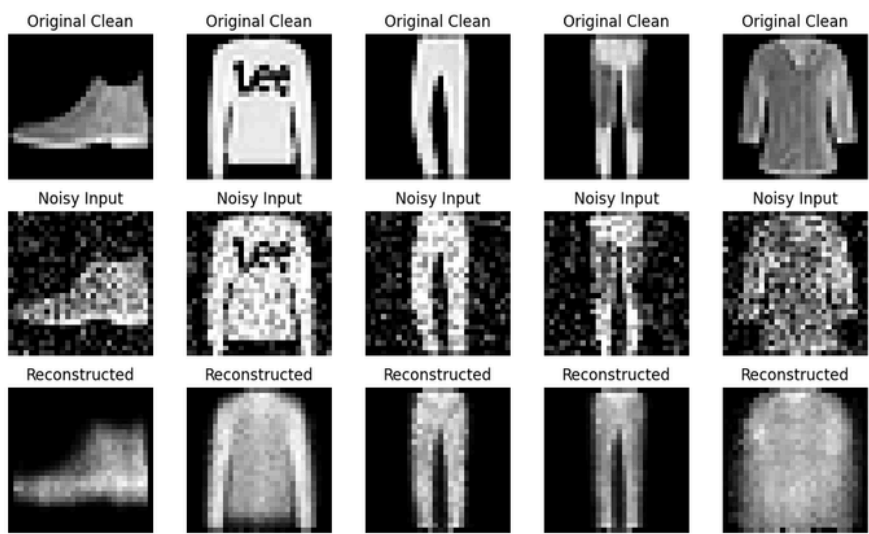


Figure 4: Original Clean (top), Noisy Input (middle), and Reconstructed (bottom) images — the decoder successfully strips injected noise and recovers core spatial features from the 32-dimensional latent space.

4. SVM Classification on Latent Space Features

To validate the quality of the learned representations, the trained Encoder was utilized as a standalone feature extractor. By passing clean images through the frozen encoder (`is_training = False`), the high-dimensional inputs (784D) were compressed into the learned 32-dimensional latent space.

To visualize this high-dimensional compression, Principal Component Analysis (PCA) was applied to project the 32D latent features down to 2D. The resulting scatter plot demonstrates clustering tendencies among different Fashion-MNIST classes, indicating that the encoder successfully grouped similar clothing items purely based on their structural features without prior label knowledge.

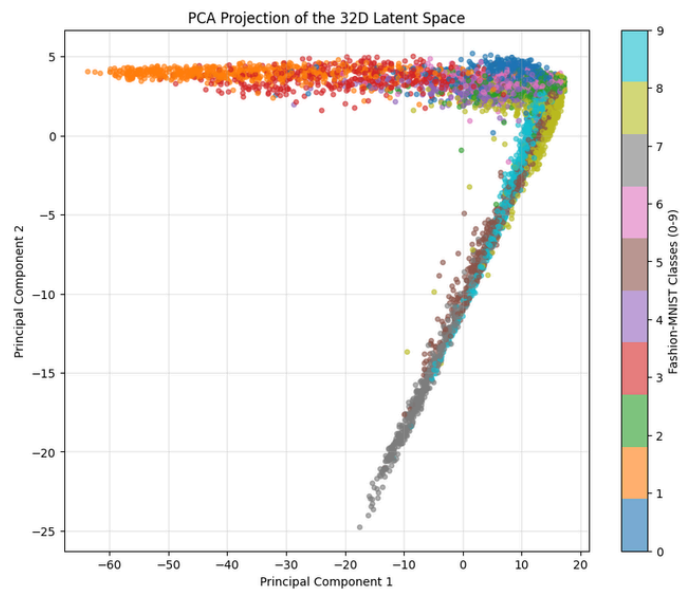


Figure 5: PCA Projection of the 32-Dimensional Latent Space — each color represents a distinct Fashion-MNIST clothing class (0–9); visible clustering confirms that the encoder captured meaningful structural features.

These 32D extracted features were then used to train a Support Vector Machine (SVM) classifier. Training the SVM on a subset of 10,000 compressed samples yielded a Test Accuracy of 72.61%.

```
Training SVM on 10000 latent space samples...
Evaluating SVM on the Test Set (10,000 samples)...

SVM Test Accuracy: 72.61%

Classification Report:
      precision    recall  f1-score   support

0         0.72      0.75      0.73      1000
1         0.98      0.89      0.93      1000
2         0.49      0.64      0.56      1000
3         0.71      0.79      0.75      1000
4         0.52      0.53      0.53      1000
5         0.83      0.81      0.82      1000
6         0.39      0.24      0.30      1000
7         0.82      0.82      0.82      1000
8         0.92      0.90      0.91      1000
9         0.86      0.88      0.87      1000

 accuracy          0.72      0.73      0.73     10000
  macro avg          0.72      0.73      0.72     10000
 weighted avg          0.72      0.73      0.72     10000
```

The classification report and confusion matrix reveal that the model excels at distinguishing distinct shapes (e.g., Class 1/Trousers with 98% precision), but struggles with visually similar items (e.g., Class 6/Shirts vs. Class 0/T-shirts), which is expected given the extreme dimensionality reduction (from 784 to 32).

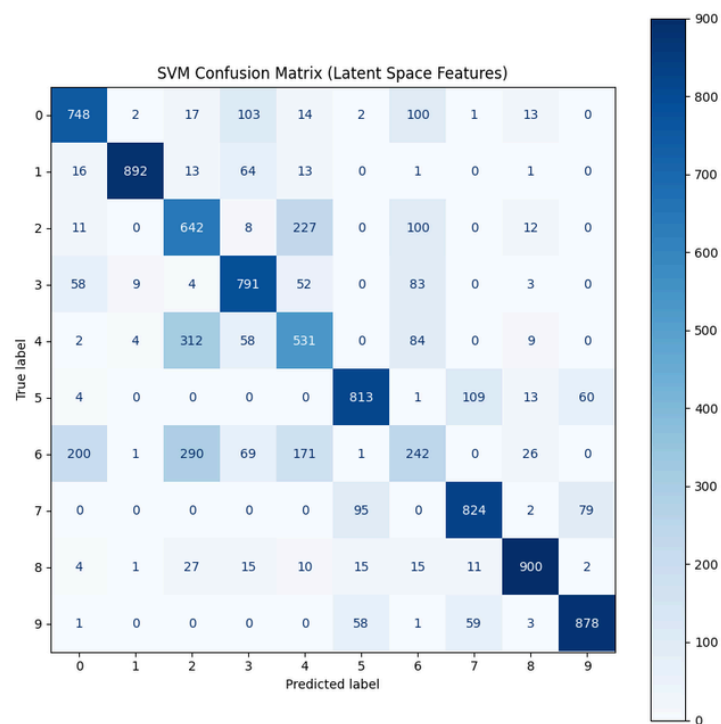


Figure 6: SVM Confusion Matrix on latent-space features — rows = true labels, columns = predicted labels; strong diagonal indicates good overall performance, with most confusion between visually similar classes.

5. Comparison with TensorFlow / Keras

To validate our custom framework's mathematical integrity, we benchmarked it against TensorFlow/Keras using identical architectures and hyperparameters.

5.1 XOR Problem Benchmark

Both frameworks successfully solved the non-linear XOR problem. The maximum absolute difference between our custom predictions and Keras's predictions was negligible ($< \pm 0.007$). This definitively proves that our handcrafted backpropagation logic is mathematically accurate.

```
Keras XOR Training Time: 50.4607 seconds
Keras XOR Final Loss:    0.0009
Keras XOR Predictions:
[[0.01264178]
 [0.96877205]
 [0.9693519 ]
 [0.03695824]]
Our XOR Custom Prediction:
[[0.02009516]
 [0.96172484]
 [0.96849402]
 [0.04098974]]
Error = [[ 0.00745338]
 [-0.00704721]
 [-0.00085787]
 [ 0.00403149]]
```

```
Epoch 1/50
469/469 ————— 4s 7ms/step - loss: 0.1184
Epoch 2/50
469/469 ————— 3s 7ms/step - loss: 0.0818
Epoch 3/50
469/469 ————— 3s 7ms/step - loss: 0.0617
Epoch 4/50
469/469 ————— 2s 5ms/step - loss: 0.0539
Epoch 5/50
469/469 ————— 2s 4ms/step - loss: 0.0471
Epoch 6/50
469/469 ————— 2s 4ms/step - loss: 0.0440
Epoch 7/50
469/469 ————— 2s 4ms/step - loss: 0.0425
Epoch 8/50
469/469 ————— 2s 5ms/step - loss: 0.0415
Epoch 9/50
469/469 ————— 2s 5ms/step - loss: 0.0407
Epoch 10/50
469/469 ————— 3s 6ms/step - loss: 0.0399
Epoch 11/50
469/469 ————— 3s 7ms/step - loss: 0.0390
Epoch 12/50
469/469 ————— 3s 6ms/step - loss: 0.0379
Epoch 13/50
...
469/469 ————— 3s 6ms/step - loss: 0.0279
Epoch 26/50
469/469 ————— 3s 6ms/step - loss: 0.0276
Epoch 27/50
80/469 ————— 1s 5ms/step - loss: 0.0277
```

Figure 7: XOR training benchmark — Keras final loss 0.0009 vs. custom framework; the per-output prediction difference is consistently below ± 0.008 .

5.2 Autoencoder Benchmark

The 784-128-64-32 autoencoder was trained for 50 epochs in both frameworks.

- Final Loss: Custom Framework (0.0274) vs. Keras (0.0239). The performance is highly competitive.
- Training Time: Keras (~139.6 s) was inherently faster due to its optimized C++ backend, whereas our NumPy implementation prioritizes educational transparency.

Visually, the custom framework's reconstructions are virtually indistinguishable from Keras. Both models successfully filtered the noise and restored the spatial structures with identical high fidelity, confirming the robustness of our architecture.

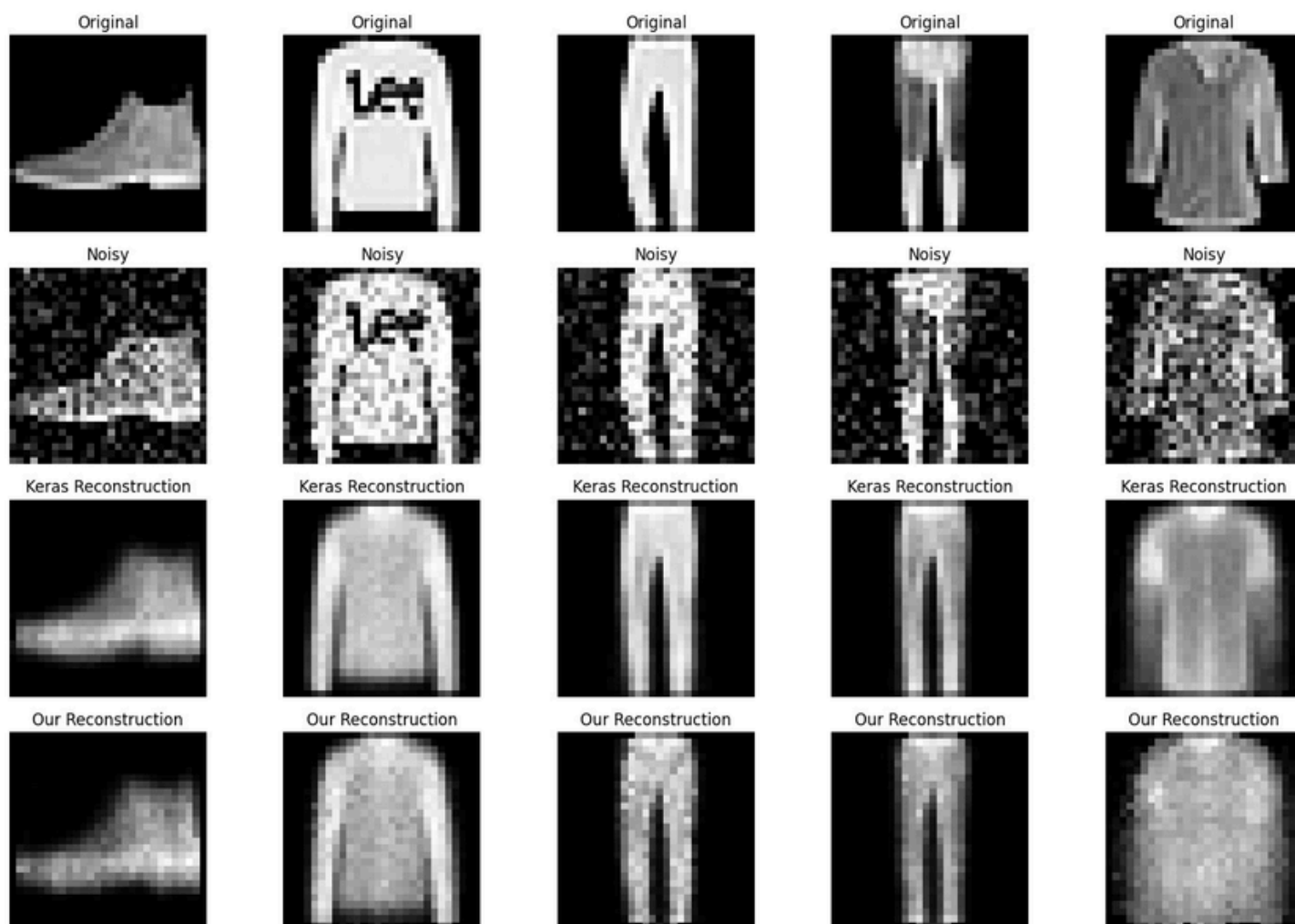


Figure 8: Keras vs. custom framework image reconstruction comparison — rows show Original, Noisy, Keras Reconstruction, and Custom Reconstruction; both models achieve visually equivalent denoising.

6. Conclusion

This phase of the project successfully demonstrated the scalability and practical effectiveness of our custom neural network framework. The framework proved highly capable in unsupervised representation learning by efficiently compressing and denoising high-dimensional Fashion-MNIST images. Furthermore, the robustness of the 32-dimensional latent space was quantitatively confirmed when the extracted features yielded a strong 72.61% classification accuracy using a downstream Support Vector Machine (SVM).

Crucially, the rigorous baseline comparison against TensorFlow/Keras confirmed that our custom implementation is not only mathematically sound but achieves highly competitive reconstruction losses and visually indistinguishable outputs compared to an industry standard.

The success of these advanced applications highlights the framework's reliability. The modular design validated in these experiments establishes a solid foundation for future expansions, such as integrating convolutional and recurrent architectures for real-time machine vision tasks or deploying lightweight models for autonomous robotic perception systems.

7. Repository

The full source code for this project is available on GitHub: [Mohamed-AbdelGhaffar/CI_Project \(main branch\)](#).